

VPS From Zero: Provision an Ubuntu Server for Hosting

Take a brand-new Ubuntu VPS from "the provider just emailed you root credentials" to "locked down, patched, with Docker Engine and a shared Postgres ready to host containerized apps." Companion document to [dockerized-deployments](#) — this one handles the foundation; that one handles the apps.

NOTE. LAST VALIDATED: 2026-05. Tool versions reflect current stable: Ubuntu 24.04 LTS, Docker Engine current, Postgres 16, latest UFW. Bump and re-validate annually per CLAUDE.md standard #5.

What you'll have at the end

- A locked-down Ubuntu 24.04 VPS with key-only SSH, root login disabled, and an active firewall.
- Two distinct identities on the box:
- **You** (an admin user with sudo) — for human work.
- A **deploy** user (docker access only, no sudo) — for CI/CD only.
- Docker Engine installed and working.
- A shared Docker network all your apps will join.
- A shared Postgres container, running with per-app users and a nightly backup cron.
- A clean directory layout for adding apps.
- Automatic security patches running unattended.

You will **not** yet have any apps running. That's the [dockerized-deployments](#) guide's job.

Prerequisites

- A Ubuntu 24.04 VPS (Hetzner, DigitalOcean, Linode, Vultr — any provider with root SSH access)
 - A laptop with WSL2 + Ubuntu (this guide is written for that environment; macOS/Linux substitute equivalents)
 - ~30 minutes start to finish the first time
-

Table of contents

1. [The principles](#)
2. [SSH key strategy: one key, every environment](#)
3. [First login to the VPS](#)
4. [System update, hostname, timezone](#)
5. [Create your admin user](#)

6. [Install your public key on the VPS](#)
 7. [Lock down SSH](#)
 8. [Firewall \(UFW\)](#)
 9. [Automatic security patches](#)
 10. [Install Docker Engine](#)
 11. [Create the deploy user \(for CI\)](#)
 12. [The CI deploy key](#)
 13. [Shared Docker network and directory structure](#)
 14. [Shared Postgres](#)
 15. [Per-app database users](#)
 16. [Database backups](#)
 17. [SSH config on your laptop](#)
 18. [Quality-of-life touches](#)
 19. [User audit checklist](#)
 20. [Troubleshooting](#)
 21. [Alternatives considered](#)
 22. [Quick reference](#)
-

1. The principles

Five ideas this whole setup rests on. Internalize them and the rest is implementation detail.

1. **Separation of identity.** You SSH in as an admin user who can `sudo`. CI SSHes in as a sandboxed `deploy` user that has Docker access but no sudo. Apps connect to Postgres as per-app users that own only their own database. If any one identity is compromised, the others contain the blast radius.
2. **Keys, not passwords.** Password authentication on a server is brute-forced within minutes of going online. Key-only auth makes that attack vector vanish. The cost is that you have to keep your private key safe — which you'd be doing anyway.
3. **Default deny on the firewall.** The only port open inbound is SSH. Add more (80/443 for HTTP) only when you have an actual service to expose. Most VPS compromises are services left listening on `0.0.0.0` with default credentials.
4. **Patch automatically.** Set up unattended security updates on day one. The number-one reason VPSes get owned is unpatched CVEs in OpenSSH, the kernel, or web servers.
5. **One shared network and one shared database.** This is what makes "add a 4th app" cost ~10 minutes instead of a weekend. You provision the foundation once; apps plug into it as they arrive.

Why this architecture vs. alternatives

Pattern	Verdict	Why
Fly.io / Railway / Render	Use these if you want zero-ops. Free tiers are tight.	Best simplicity; monthly costs scale per-app.
Heroku-style PaaS on your VPS (Coolify, Dokploy)	Reasonable upgrade if/when ops becomes a burden.	Web UI, less to learn upfront, but a heavier abstraction to debug when it breaks.
Kubernetes (k3s, k0s)	Overkill for a personal VPS.	The complexity tax isn't justified until you have ≥ 5 nodes or genuine HA needs.
Plain systemd + Python venv on the host	Works, but loses "same image in dev and prod."	Migrating between machines is painful; toolchain drift accumulates.
This guide	The sweet spot for 1-10 apps on one box.	Add one app at a time without disturbing others; same workflow scales.

2. SSH key strategy: one key, every environment

Goal

Generate **one** Ed25519 keypair. Use it from WSL, from Windows-native tools (PowerShell, VS Code on Windows, GitHub Desktop), and from inside any Linux container that needs SSH. Don't proliferate keys; that way lies "wait, which key does this machine know?"

There's a **separate** key for CI (Step 12). That key is intentionally different — narrower scope, no passphrase, never on your laptop's SSH agent.

Why WSL is the right canonical home

- **Performance.** ext4 honors strict file permissions; SSH refuses keys with sloppy perms. The 9p mount of `/mnt/c` doesn't behave well here.
- **Where you actually work.** Day-to-day terminal work happens in WSL.
- **Dev container forwarding.** VS Code Dev Containers automatically forward your WSL `ssh-agent` socket into any container they build, with zero config.
- **Backup story.** You can mirror the keypair to `C:\Users\<you>\.ssh\` so it survives a WSL distro reinstall.

Step 2.1 — Generate the key

In WSL:

```
ssh-keygen -t ed25519 -C "you@example.com"
# accept default path: ~/.ssh/id_ed25519
# set a passphrase – yes, really
```

Why Ed25519? Smaller, faster, more secure than RSA. Supported by every modern SSH server and GitHub. Don't use RSA in 2026.

Why a passphrase? Defense in depth. If your laptop is stolen with the disk decrypted, the attacker still needs your passphrase to use the key. `ssh-agent` caches the unlocked key for the session, so you type the passphrase once.

Step 2.2 — Persistent ssh-agent in WSL

Without persistence, every new terminal asks for your passphrase. Install `keychain`:

```
sudo apt install keychain
```

Add to `~/.bashrc`:

```
eval "$(keychain --eval --quiet id_ed25519)"
```

Open a new terminal, type the passphrase once when prompted, and every subsequent terminal reuses the same agent.

Why `keychain` over raw `ssh-agent`? - Raw `ssh-agent` starts a new agent per shell unless you manually track the socket path. - `keychain` finds an existing agent and reuses it, or starts a new one only if needed. - Survives terminal close/reopen; only dies on system reboot.

Alternative — 1Password SSH Agent. If you already use 1Password, its built-in SSH agent is genuinely better: key never touches disk, syncs across devices, biometric auth per use. Skip `keychain` and point `SSH_AUTH_SOCK` at 1Password's socket. Worth it if you're already paying for 1Password.

Step 2.3 — Mirror to Windows (for backup + native tools)

```
# in WSL
mkdir -p /mnt/c/Users/<YOUR_WINDOWS_USERNAME>/.ssh
cp ~/.ssh/id_ed25519 ~/.ssh/id_ed25519.pub /mnt/c/Users/<YOUR_WINDOWS_USERNAME>/.ssh/
```

Then in **PowerShell as administrator** (one time):

```
Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent
ssh-add $env:USERPROFILE\.ssh\id_ed25519
```

Now Git for Windows, GitHub Desktop, VS Code on Windows, and `git push` from PowerShell all find and use the same key. And if you ever blow away your WSL distro, the key still exists on Windows.

Step 2.4 — Add the public key to GitHub

```
cat ~/.ssh/id_ed25519.pub
```

Copy the output. Go to github.com/settings/keys → "New SSH key" → paste. Title it something like "WSL on laptop, 2026".

Test:

```
ssh -T git@github.com
# expect: "Hi <username>! You've successfully authenticated..."
```

3. First login to the VPS

Your provider has given you an IP and either a root password or asked you to paste a key during provisioning.

If your provider lets you paste a key during provisioning, paste your `~/.ssh/id_ed25519.pub` and you can skip ahead to [Step 5](#) (still logged in as root via key).

Otherwise, log in with the provided password:

```
ssh root@<vps-ip>
# enter the provider's initial password
```

4. System update, hostname, timezone

```
apt update && apt upgrade -y
hostnamectl set-hostname my-vps          # whatever you want to call this box
timedatectl set-timezone America/Chicago  # or your timezone
```

Why upgrade first? Provider images are often weeks out of date. Patching is the highest-leverage security action you can take in the first five minutes. `unattended-upgrades` (Step 9) keeps you patched going forward.

Why set the hostname? When you SSH in months from now, `whoami@my-vps` is more useful than `whoami@ubuntu-22-04-x64-fra1-01`. It also shows up in monitoring tools and Postgres logs.

Why set the timezone? Cron jobs (like the nightly backup we set up in Step 16) run on the system's local time. If you put backups at 3am Central, you want 3am Central, not 3am UTC.

5. Create your admin user

```
adduser julian          # prompts for password; used only for sudo escalation
usermod -aG sudo julian
```

Why a separate user instead of using root?

- A momentary `rm -rf` typo as root is catastrophic. As `julian`, sudo asks for a password and gives you a half-second to think.
- Audit logs distinguish "what did the human do" from "what did the system do."
- Standard Unix hygiene — every distro and every guide assumes you do this.

The user gets a password (for sudo prompts), but **doesn't use it for SSH login** — keys handle that.

6. Install your public key on the VPS

From a **new WSL terminal** (keep the root session open as a safety net):

```
ssh-copy-id julian@<vps-ip>
# enter julian's password once

ssh julian@<vps-ip>
# should log in without password
```

Why `ssh-copy-id` instead of doing it by hand? It appends your public key to `/home/julian/.ssh/authorized_keys` and fixes the permissions correctly. `authorized_keys` must be `0600` and `~/.ssh` must be `0700`, or SSH silently refuses the key. People get this wrong by hand all the time.

7. Lock down SSH

As `julian` on the VPS:

```
sudo nano /etc/ssh/sshd_config
```

Find and set:

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
```

Then:

```
sudo systemctl restart ssh
```

WARNING. DON'T CLOSE YOUR CURRENT SESSION YET. Open a new WSL terminal and confirm `ssh julian@<vps-ip>` still works. If it does, you're safe to disconnect everything. If it doesn't, you still have the original session to fix it. Misconfigured `sshd` is the #1 way personal VPSes get locked out forever — the box is fine but nobody can get in.

What each setting does:

- `PermitRootLogin no` — kills the root SSH brute-force attack surface entirely.
 - `PasswordAuthentication no` — even if someone steals `julian`'s password, they can't SSH in with it. Keys only.
 - `PubkeyAuthentication yes` — explicitly enables the only auth method left.
-

8. Firewall (UFW)

```
sudo ufw allow OpenSSH
sudo ufw enable
sudo ufw status
```

That's it. **Default-deny** for everything except SSH.

Why UFW over plain iptables? UFW is iptables with a sane interface. It's not less powerful; it's just less typo-prone. UFW rules compile down to iptables rules under the hood.

When do you add more? When you actually run a service that needs an inbound port. The dockerized-deployments guide adds `80/tcp` and `443/tcp` in the Caddy step. Postgres stays internal-only — never expose it.

9. Automatic security patches

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades
# select "Yes" when prompted
```

What this does: Installs **security-only** patches automatically (no feature upgrades, no breaking changes). The box reboots itself in the middle of the night only if a kernel update requires it. Configurable in `/etc/apt/apt.conf.d/50unattended-upgrades`.

Why security-only and not all updates? Feature upgrades sometimes break compatibility. You want them on your schedule, not the package maintainer's. Security patches are different — those you want immediately.

10. Install Docker Engine

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker julian
# log out and back in for group to take effect
exit
ssh julian@<vps-ip>
docker run hello-world           # should succeed without sudo
```

Why the official install script and not `apt install docker.io`? The Ubuntu repo's `docker.io` is often months behind. The official script adds Docker's apt repo and pulls the latest stable version, which you'll want for security and modern features like BuildKit-by-default.

Why add yourself to the docker group? So you don't have to `sudo docker` every time.

CAUTION. ANYONE IN THE DOCKER GROUP CAN EFFECTIVELY BECOME ROOT — they can mount the host filesystem into a privileged container. This is a known property of Docker, not a bug. Mitigation: only put accounts you trust in the docker group (you and the deploy user). For higher-paranoia setups, use rootless Docker — but that's a separate guide.

11. Create the deploy user (for CI)

```
sudo adduser --disabled-password --gecos "" deploy
sudo usermod -aG docker deploy
```

What the flags mean:

- `--disabled-password` — no password is set. The user can only log in with an SSH key (which we install in Step 12). A user with no password can't be brute-forced over SSH even if password auth gets re-enabled by accident.
- `--gecos ""` — skip the interactive "Full Name / Room Number / Phone" prompts. Those fields are remnants from the 1970s and no one uses them.

What the user can and can't do:

- ☐ Run `docker` commands (in the `docker` group)
- ☐ SSH in with the CI key (set up in Step 12)
- ☒ `sudo` (not in `sudoers`)
- ☒ Log in with a password (no password set)

Why a second user and not just reuse `julian`? Because the CI key has to live in a GitHub secret, which means anyone who compromises that secret gets whatever the user can do. By making it a sandboxed user with no `sudo`, the worst case is "attacker can mess with containers" — not "attacker has full root on the box."

12. The CI deploy key

Generate this **on your laptop**, not on the VPS. The private half will live in GitHub's Secrets, never on your local SSH agent (so you can't accidentally use it for personal SSHing).

```
ssh-keygen -t ed25519 -f ~/.ssh/vps-deploy -C "github-actions deploy key" -N ""
```

`-N ""` means no passphrase. CI can't type one. The reduced security is mitigated by:

- The key is single-purpose (deploy user only, no `sudo`).
- The key only exists in two places: encrypted in GitHub Secrets, and `authorized_keys` on the VPS.
- You can revoke it instantly by deleting it from `/home/deploy/.ssh/authorized_keys` on the VPS.

Install the public half on the VPS

```
# from your laptop
scp ~/.ssh/vps-deploy.pub julian@<vps-ip>:/tmp/

# on the VPS, as julian
sudo mkdir -p /home/deploy/.ssh
sudo cp /tmp/vps-deploy.pub /home/deploy/.ssh/authorized_keys
sudo chown -R deploy:deploy /home/deploy/.ssh
sudo chmod 700 /home/deploy/.ssh
sudo chmod 600 /home/deploy/.ssh/authorized_keys
```


Test from your laptop

```
ssh -i ~/.ssh/vps-deploy deploy@<vps-ip> "docker ps"
# should succeed and show running containers (probably none yet)
```

The private key file (`~/.ssh/vps-deploy`) will be pasted into GitHub Secrets as `VPS_SSH_KEY` later when you set up CI in the `dockerized-deployments` guide.

13. Shared Docker network and directory structure

Switch to the `deploy` user (or `sudo -iu deploy`) on the VPS:

```
sudo -iu deploy
docker network create shared
mkdir -p ~/infra
mkdir -p ~/<your-first-app-name>
```

Why one shared network created by hand?

By default, `docker compose up` creates its own network scoped to that compose project. Containers on different networks can't talk to each other. We want our apps to be able to reach the shared Postgres, so they all need to be on the same network — one that exists independently of any compose project.

Every app's compose file will declare:

```
networks:
  shared:
    external: true
```

The `external: true` is the key part — it means "this network already exists; just connect to it; don't try to create it."

Why the directory layout (`infra/` plus one folder per app)? It maps directly onto the compose-stack model. The `infra/` directory holds the `docker-compose.yml` for shared services (Postgres, Caddy, etc.). Each app gets its own directory with its own `docker-compose.yml`. Each stack is independent — restarting one doesn't touch the others.

14. Shared Postgres

Still as `deploy`, create `~/infra/docker-compose.yml`:

```

networks:
  shared:
    external: true

services:
  postgres:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_ROOT_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - shared
    # NOTE: no `ports:` block – Postgres is NOT exposed to the internet.
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres_data:

```

And `~/infra/.env`:

```
POSTGRES_ROOT_PASSWORD=<long-random-string>
```

Generate the password:

```
openssl rand -base64 32
```

Then bring it up:

```

cd ~/infra
docker compose up -d
docker compose ps          # confirm "healthy"
docker compose logs postgres

```

What each line of this compose file does, and why

image: postgres:16 Pin a specific major version. **:latest is dangerous here** — Postgres major versions are not data-file-compatible. If a `docker compose pull` upgrades you from 16 to 17 across a reboot, the new container will fail to start because the data files are in the 16 format. With the major version pinned, you upgrade on your schedule (with a dump-and-restore).

restart: unless-stopped Survives reboots and crashes — Docker brings the container back up automatically. The "unless-stopped" part means a manual `docker compose stop` actually stays stopped. Compare with `restart: always`, which fights you by restarting after a manual stop. `unless-stopped` is almost always what you want.

environment.POSTGRES_PASSWORD: \${POSTGRES_ROOT_PASSWORD} The official `postgres` image reads this at first startup and uses it as the password for the `postgres` superuser. The `${...}` syntax pulls from the `.env` file in the same directory. **Keep this password out of git** — `.env` should be gitignored if you ever version-control the infra directory.

volumes: postgres_data:/var/lib/postgresql/data Database files persist in a Docker-managed named volume (`postgres_data`). If you didn't have this volume, the data would live in the container's filesystem and disappear the moment the container is destroyed. Named volumes survive `docker compose down`; they're only deleted by `docker compose down -v` or `docker volume rm`. We chose a named volume rather than a host bind mount because:

- Docker manages the permissions correctly (Postgres is picky about file ownership).
- It's easy to back up with `docker exec ... pg_dumpall`.
- It's harder to accidentally delete (a bind mount under `~/data/` can be `rm -rf`'d by a typo).

No ports: block This is the most important security setting in the whole file. With no `ports:` declared, Postgres listens only on the Docker bridge network — not on the host's public IP. The only way to reach it is from another container on the `shared` network. If we had said `ports: ["5432:5432"]`, Postgres would be on `0.0.0.0:5432` of the VPS, exposed to the internet, where brute-force bots would start hitting it within minutes.

networks: [shared] Attach to the shared network we created in Step 13. Other containers on the same network can reach this one by its service name (`postgres`) as the hostname.

healthcheck `pg_isready` is a Postgres tool that returns success when the server is accepting connections. Docker reports the container as "healthy" only when the healthcheck passes. App containers can then say `depends_on: postgres: condition: service_healthy` to wait until Postgres is actually serving queries — not just "the container started." Without this, apps would race the database on startup and crash on their first query.

15. Per-app database users

Connect to the shared Postgres:

```
docker compose exec postgres psql -U postgres
```

For each app, create a user and a database:

```
CREATE USER appuser WITH PASSWORD 'long-random-string';
CREATE DATABASE appdb OWNER appuser;
GRANT ALL PRIVILEGES ON DATABASE appdb TO appuser;
\q
```

Why per-app users (and not "just use the postgres superuser")

Defense in depth. If an app's `.env` file leaks (committed by accident, server compromised, etc.), the attacker gets access to:

- That app's database, full read/write
- Nothing else

Compared with reusing the superuser, where a leak gives the attacker **every database on the server**, plus the ability to drop users, create new databases, etc.

The cost is one `CREATE USER` + `CREATE DATABASE` per app — about ten seconds.

Password generation

Don't pick passwords by hand. Run:

```
openssl rand -base64 24
```

Save the result to two places:

1. The app's `.env` on the VPS (`/home/deploy/<app>/ .env`).
2. A password manager (1Password, Bitwarden, etc.) — your long-term recovery.

Never commit the password to git, and never reuse it across apps.

Auditing what's there

Once you have multiple apps, periodically audit users and databases:

```
\du                -- list all users + their attributes
\du+ appuser       -- detailed info on one user
\l                -- list all databases + owners
\l+ appdb          -- detailed info on one database
```

What you want to see:

- Each app user owns exactly one database.
- No app user has the `Superuser`, `Create role`, or `Create DB` attributes.
- The `postgres` user is the only one with elevated attributes.

Read-only user (for analytics / debugging)

When you want to poke at production data without risk of mutation:

```
CREATE USER app_readonly WITH PASSWORD '<random>';
GRANT CONNECT ON DATABASE appdb TO app_readonly;
\c appdb
GRANT USAGE ON SCHEMA public TO app_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO app_readonly;
```

The `ALTER DEFAULT PRIVILEGES` line is the important one: it ensures tables added by future migrations are automatically readable, so you don't have to re-grant after every schema change.

16. Database backups

A nightly cron that `pg_dumpall`s and optionally rsyncs offsite.

As `julian` (or root), create `/etc/cron.d/pg-backup`:

```
0 3 * * * deploy /home/deploy/scripts/backup-db.sh
```

Then as `deploy`, create `/home/deploy/scripts/backup-db.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

BACKUP_DIR=/home/deploy/backups
mkdir -p "$BACKUP_DIR"

TIMESTAMP=$(date +%Y%m%d-%H%M%S)
FILE="$BACKUP_DIR/pgdump-$TIMESTAMP.sql.gz"

docker exec infra-postgres-1 pg_dumpall -U postgres | gzip > "$FILE"

# Keep last 14 days locally
find "$BACKUP_DIR" -name 'pgdump-*.sql.gz' -mtime +14 -delete

# Optional: rsync offsite
# rsync "$FILE" backup-server:~/pgdumps/
```

Make it executable:

```
chmod +x /home/deploy/scripts/backup-db.sh
```

Why a script + cron, not a backup-in-the-container

The backup runs outside the Postgres container so it has access to the host's filesystem (where the backup files end up). `docker exec` runs `pg_dumpall` inside the running Postgres container, but the output (via the pipe) flows to the host. This is the simplest setup that gets the data off the in-container volume.

Why `pg_dumpall` and not `pg_dump`

`pg_dumpall` dumps **all databases and global state** (users, passwords, roles). `pg_dump` dumps a single database. For a personal box with a handful of small databases, dumping everything is cheaper than figuring out which databases are important.

Why gzip the output

Postgres dumps are extremely compressible (lots of repeated SQL). A 100MB dump might compress to 10MB. Disk is cheap but offsite bandwidth isn't always.

Offsite storage

Backblaze B2 at \$0.005/GB/month is the cheapest credible offsite option. `rclone` syncs the local backup directory to B2. **Don't trust a backup that lives on the same machine as the database** — disk failure kills both.

Restoring from a backup

```
gunzip < pgdump-YYYYMMDD-HHMMSS.sql.gz | docker exec -i infra-postgres-1 psql -U postgres
```

Test the restore path once, before you ever need it. A backup that hasn't been successfully restored is a wish, not a backup.

17. SSH config on your laptop

Edit `~/.ssh/config` in WSL:

```
Host myvps
  HostName <vps-ip-or-domain>
  User julian
  IdentityFile ~/.ssh/id_ed25519

Host myvps-deploy
  HostName <vps-ip-or-domain>
  User deploy
  IdentityFile ~/.ssh/vps-deploy
```

Now `ssh myvps` is your admin login; `ssh myvps-deploy` lets you SSH in as the deploy user (for debugging CI issues).

Why two entries? Same IP, different user + different key. The config is just a shortcut so you don't have to type `-i ~/.ssh/vps-deploy deploy@<vps-ip>` every time.

18. Quality-of-life touches

As `julian` on the VPS:

```
# helpful aliases
cat >> ~/.bashrc << 'EOF'
alias dc='docker compose'
alias dlogs='docker compose logs -f --tail=100'
alias dexec='docker compose exec'
alias dps='docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Image}}"'
EOF

# debug tools you'll wish you had at 11pm
sudo apt install -y htop ncd u jq tree
```

Why these specifically?

- `htop` — interactive process viewer. Much better than `top` for tracking down what's eating CPU/memory.
 - `ncdu` — interactive disk usage. When `df` says you're full, `ncdu` shows you which directory tree to clean up.
 - `jq` — JSON parser. Useful for the inevitable moment you need to grep through a JSON log file or API response.
 - `tree` — recursive directory listing. Quick way to see project structure.
-

19. User audit checklist

Run quarterly to catch drift:

- [] `cat /etc/passwd` — any users you don't recognize?
 - [] `groups julian` + `groups deploy` — correct group memberships?
 - [] `sudo cat /home/julian/.ssh/authorized_keys` — only your keys, no surprises?
 - [] `sudo cat /home/deploy/.ssh/authorized_keys` — only the current CI deploy key?
 - [] `last -a | head -20` — any logins from IPs you don't recognize?
 - [] `journalctl -u ssh -n 100 | grep "Failed password"` — any brute-force attempts? (With password auth disabled, these should all fail harmlessly.)
 - [] `sudo grep -r '' /etc/sudoers /etc/sudoers.d/` — only `julian` and `%sudo` group?
 - [] In `psql`: `\du` — only expected app users, none with elevated privileges?
-

20. Troubleshooting

"Permission denied (publickey)" when SSHing as julian

Common causes, in order of likelihood:

1. **Wrong permissions on `~/.ssh/id_ed25519`** — must be `600`. `chmod 600 ~/.ssh/id_ed25519`.
2. **Public key not installed correctly on VPS.** On VPS: `cat ~/.ssh/authorized_keys` and confirm your key is the only one there.
3. **sshd_config disabled key auth.** `sudo grep PubkeyAuthentication /etc/ssh/sshd_config` should be `yes`.
4. **Agent not loaded.** `ssh-add -L` should show your public key. If empty, run `eval $(keychain --eval id_ed25519)`.

"Permission denied" when running docker commands

You're not in the docker group, or the group hasn't taken effect yet:

```
groups    # confirm 'docker' is listed
# if not:
sudo usermod -aG docker $USER
# log out completely and back in (not just close terminal – close session)
```

"Cannot start service postgres" after a Postgres major-version bump

You upgraded `postgres:16` → `postgres:17` (or used `:latest`) without doing a dump/restore.

Recovery:

1. Revert to `postgres:16` in `infra/docker-compose.yml`, run `docker compose up -d`.
2. `docker exec infra-postgres-1 pg_dumpall -U postgres > dump.sql`.
3. `docker compose down` + delete the volume: `docker volume rm infra_postgres_data`.
4. Update to `postgres:17`, run `docker compose up -d` (creates a fresh empty volume).
5. Restore: `docker exec -i infra-postgres-1 psql -U postgres < dump.sql`.

Or just pin major versions and don't bump without planning.

"Could not resolve hostname" on `docker compose pull`

The container can't reach the registry. Either:

- The VPS doesn't have outbound internet (provider firewall?). Test with `curl https://google.com`.
- DNS is broken inside the container. Test with `docker run --rm alpine nslookup github.com`.
- You haven't logged the deploy user into the registry yet. See the dockerized-deployments guide for `docker login ghcr.io`.

Disk filling up

Docker's number-one way to ruin your week is filling the disk with unused images, build cache, and stopped containers. Once a week:

```
docker system df          # see what's using space
docker system prune -af   # aggressively clean up unused everything
docker volume prune       # be careful – won't delete in-use volumes, but will delete stopped-
                           # app volumes
```

Add a weekly cron if you don't want to remember.

21. Alternatives considered

Hosting choices we rejected

Fly.io / Railway / Render - Why not: Per-app monthly cost adds up; learning the abstraction is non-portable. - When to reconsider: You're hosting only 1-2 apps and your time is worth more than \$5-20/month/app.

Kubernetes (k3s, k0s, single-node) - Why not: The complexity tax is enormous and the payoff is multi-node orchestration you don't need on a single VPS. - When to reconsider: You actually have ≥ 3 nodes, real HA requirements, or you're learning K8s as a skill.

Coolify, Dokploy, CapRover - Why not for this guide: Hides the docker/compose layer we explicitly want you to understand. When something breaks at the compose layer, you have to break through the abstraction to debug. - When to reconsider: You've internalized the docker-compose flow, you're running 5+ apps, and the GUI ergonomics are worth the trade.

OS choices

Ubuntu 24.04 LTS (this guide) - Most documentation written for it, predictable LTS support.

Debian 12 - Equivalent capability, even more minimal, slower to add new packages.

Fedora / Rocky / Alma - RHEL family; better if you're in a corporate Red Hat shop. Different package management (`dnf`), different SELinux defaults.

SSH agent choices

keychain (this guide) - Pure bash, no extra services, survives terminal close.

1Password SSH Agent - Biometric per-use auth, syncs across devices. Costs \$\$\$. Best UX bar none.

Windows OpenSSH agent + npiperelay - Single agent shared with Windows. Janky setup with multiple bridge tools.

No agent, type passphrase each time - Don't. You'll disable the passphrase out of frustration within a week.

22. Quick reference

Connect to the VPS

```
ssh myvps          # admin user (julian)
ssh myvps-deploy   # deploy user (debugging CI)
```

Manage the shared infra stack

```
cd ~/infra
docker compose up -d      # bring it up
docker compose ps         # status
docker compose logs postgres # logs
docker compose down       # stop (data persists in volume)
```

Manage Postgres users + databases

```
docker compose -f ~/infra/docker-compose.yml exec postgres psql -U postgres
```

In psql:

```
\du          -- list users
\l           -- list databases
\c appdb     -- connect to a database
\dt          -- list tables in current db
CREATE USER ...;
CREATE DATABASE ...;
ALTER USER ... WITH PASSWORD '...';
```

Backup + restore

```
# manual backup
docker exec infra-postgres-1 pg_dumpall -U postgres | gzip > backup-$(date +%F).sql.gz

# restore
gunzip < backup.sql.gz | docker exec -i infra-postgres-1 psql -U postgres
```

Disk + container hygiene

```
docker system df          # what's using space
docker system prune -af   # clean everything unused
docker volume prune       # clean dangling volumes (careful)
ncdu ~                    # explore disk usage
```

Next

You now have a server ready to host containerized apps. Continue with [dockerized-deployments](#) to wire up the CI/CD pipeline and ship your first app.